Desglose Detallado Clase por Clase

Clase 1: El Ecosistema HTTP y el Esqueleto Clean Architecture (2 hrs)

- **Objetivo específico:** Que el alumno comprenda cómo se comunica una API mediante HTTP y construya la solución en blanco entendiendo por qué las capas están separadas.
- **Desglose de tiempos:**
    - **0:00 - 0:40 (Teoría):** Qué es una API RESTful. Anatomía de una petición HTTP (Verbos GET, POST, PUT, DELETE, Headers, Body JSON y Status Codes 200, 201, 400, 404, 500).
    - **0:40 - 1:15 (Arquitectura):** El problema del "código espagueti". Introducción visual a Clean Architecture: la regla de dependencia explicada con diagramas sencillos.
    - **1:15 - 2:00 (Práctica guiada):** Creación de la solución en .NET 10, los 4 proyectos (Domain, Application, Infrastructure, WebApi), configuración de referencias entre proyectos y limpieza de archivos por defecto (WeatherForecast).
- **Ejercicio práctico (Entregable 1):**
    - Crear la solución desde cero, agregar las referencias correctas y comprobar que Domain no tiene ninguna referencia hacia afuera. Levantar el proyecto WebApi vacío y visualizar la pantalla inicial de Swagger/OpenAPI.

Clase 2: Modelado del Dominio y Contratos de Acceso a Datos (2 hrs)

- **Objetivo específico:** Diseñar las entidades del negocio usando clases puras de C# (POCOs) y definir las interfaces de persistencia para entender el principio de Inversión de Dependencias.
- **Desglose de tiempos:**
    - **0:00 - 0:45 (Práctica en Domain):** Creación de una clase base BaseEntity (Id, FechaCreacion, Activo) y las entidades Categoria y Producto con sus propiedades de navegación.
    - **0:45 - 1:20 (Teoría de Interfaces):** ¿Por qué usar interfaces? Explicación sencilla de cómo Application pide datos sin saber que existe SQL Server.
    - **1:20 - 2:00 (Práctica en Application):** Creación de las interfaces IGenericRepository<T> (con métodos GetByIdAsync, GetAllAsync, AddAsync, Update, Delete) e IUnitOfWork.
- **Ejercicio práctico (Entregable 2):**

- Los alumnos deben agregar una tercera entidad sencilla al dominio (ej. Proveedor), relacionarla con Producto y definir su contrato específico IProveedorRepository en caso de requerir una búsqueda especial (ej. buscar por RFC/identificador fiscal).

Clase 3: Persistencia con EF Core, SQL Server y Migraciones (2 hrs)

- **Objetivo específico:** Configurar Entity Framework Core en la capa Infrastructure aplicando buenas prácticas de configuración separada (Fluent API) y generar la base de datos en SQL Server.
- **Desglose de tiempos:**
  - **0:00 - 0:30 (Teoría):** Qué es un ORM. Por qué evitar llenar las entidades de Domain con atributos como [Required] o [Column] (Data Annotations) y preferir IEntityTypeConfiguration<T>.
  - **0:30 - 1:15 (Práctica):** Instalación de paquetes NuGet (Microsoft.EntityFrameworkCore.SqlServer, Tools). Creación de AppDbContext y archivos de configuración para Producto (precisión decimal para precios, longitudes máximas, llaves foráneas).
  - **1:15 - 2:00 (Base de datos):** Configuración de la cadena de conexión en appsettings.Development.json, registro del DbContext y ejecución de Add-Migration y Update-Database. Revisión de las tablas creadas en SQL Server.
- **Ejercicio práctico (Entregable 3):**
  - Crear la clase de configuración ProveedorConfiguration definiendo restricciones de longitud e índices únicos, generar una nueva migración y verificar la tabla en el gestor de base de datos.

Clase 4: Implementación de Repositorio Genérico y Unit of Work (2 hrs)

- **Objetivo específico:** Implementar las clases concretas que hablan con la base de datos y comprender el manejo de transacciones mediante Unit of Work.
- **Desglose de tiempos:**
  - **0:00 - 0:30 (Teoría y Rendimiento):** Cómo funciona el *Change Tracker* de EF Core. Cuándo usar .AsNoTracking() para mejorar el rendimiento y ahorrar memoria en consultas de lectura.
  - **0:30 - 1:15 (Práctica Repositorio):** Programación de GenericRepository<T> implementando IGenericRepository<T> usando DbSet<T>.
  - **1:15 - 2:00 (Práctica Unit of Work):** Programación de la clase UnitOfWork que agrupa los repositorios y expone SaveChangesAsync(). Explicación de por qué el repositorio no debe hacer SaveChanges por su cuenta.

- **Ejercicio práctico (Entregable 4):**
    - Integrar el repositorio de Proveedor dentro del UnitOfWork e implementar un método de consulta con filtro usando expresiones Lambda y .AsNoTracking().

Clase 5: DTOs, Validaciones con FluentValidation y Servicios (2 hrs)

- **Objetivo específico:** Aislar la base de datos del exterior usando objetos de transferencia de datos (DTOs), validar reglas de entrada y programar los casos de uso en Servicios.
- **Desglose de tiempos:**
    - **0:00 - 0:35 (Seguridad y Diseño):** El peligro de exponer las entidades directamente (*Over-posting / Mass Assignment* y referencias circulares). Creación de CreateProductoDto, UpdateProductoDto y ProductoResponseDto. Mapeo manual limpio (métodos de extensión) para que entiendan qué ocurre por debajo sin depender de cajas negras.
    - **0:35 - 1:10 (FluentValidation):** Instalación de FluentValidation. Creación de CreateProductoValidator (validar que el precio sea mayor a 0, el nombre no esté vacío, etc.).
    - **1:10 - 2:00 (Servicios / Casos de Uso):** Creación de IProductoService y ProductoService. Inyección de IUnitOfWork y IValidator<T> por constructor para orquestar el flujo completo.
- **Ejercicio práctico (Entregable 5):**
    - Crear los DTOs, el validador de FluentValidation y el servicio completo (CRUD) para la entidad Categoria.

Clase 6: Controladores, Inyección de Dependencias y Swagger (2 hrs)

- **Objetivo específico:** Conectar todas las piezas en el contenedor de Inyección de Dependencias y exponer los primeros endpoints funcionales para probarlos en Swagger UI.
- **Desglose de tiempos:**
    - **0:00 - 0:35 (Inyección de Dependencias):** Explicación gráfica de Transient, Scoped y Singleton. Creación de contenedores limpios (DependencyInjection.cs en Application e Infrastructure) para no saturar Program.cs.
    - **0:35 - 1:25 (Controladores Delgados):** Creación de ProductosController. Regla de oro: *"El controlador solo recibe la petición, llama al servicio y devuelve un código HTTP"*. Uso de CreatedAtAction, Ok, NotFound y NoContent.

    - **1:25 - 2:00 (Pruebas en Swagger):** Configuración de OpenAPI/Swagger en .NET 10. Ejecución de pruebas completas de creación, lectura, actualización y borrado desde el navegador.

- **Ejercicio práctico (Entregable 6):**
    - Construir el CategoriasController, registrar su servicio en el contenedor de dependencias y probar el flujo completo en Swagger creando primero una categoría y luego un producto asociado a ella.

Clase 7: Manejo Global de Errores y Optimización (2 hrs)

- **Objetivo específico:** Eliminar bloques try-catch repetitivos en los controladores mediante un Middleware centralizado y estandarizar las respuestas de error.
- **Desglose de tiempos:**
    - **0:00 - 0:30 (Seguridad y Estándares):** Por qué nunca devolver un StackTrace de error en producción. Qué es el estándar RFC 7807 (ProblemDetails).
    - **0:30 - 1:20 (Middleware / IExceptionHandler):** Creación de excepciones personalizadas en Domain/Application (NotFoundException, ValidationException) y configuración de IExceptionHandler global en WebApi para transformarlas automáticamente en respuestas 404 o 400 limpias.
    - **1:20 - 2:00 (Buenas prácticas y Paginación):** Cómo evitar traer 1 millón de registros por accidente. Implementación rápida de paginación básica (Skip y Take) en el listado de productos.
- **Ejercicio práctico (Entregable 7):**
    - Provocar errores intencionales en Swagger (enviar un producto con precio negativo o buscar un ID inexistente) y verificar que la API responda con el formato JSON estandarizado de ProblemDetails sin colapsar.

Clase 8: Seguridad con ASP.NET Core Identity y JWT (2 hrs)

- **Objetivo específico:** Proteger los endpoints de la API mediante autenticación basada en tokens JWT utilizando ASP.NET Core Identity.
- **Desglose de tiempos:**
    - **0:00 - 0:30 (Conceptos de Seguridad):** Diferencia entre Autenticación (401 Unauthorized) y Autorización (403 Forbidden). Cómo funciona un JSON Web Token (JWT) en una API sin estado (*Stateless*).
    - **0:30 - 1:15 (Identity en Infrastructure):** Cambio de DbContext a IdentityDbContext<IdentityUser>. Migración de tablas de seguridad (AspNetUsers, AspNetRoles). Creación del servicio generador de JWT.

    - **1:15 - 2:00 (Protección y Swagger):** Creación de AuthController (Register y Login). Configuración de Swagger para aceptar el botón **Authorize (Bearer Token)**. Colocación de [Authorize] en ProductosController.

- **Ejercicio práctico (Proyecto Final del Curso):**
    - Dejar los endpoints GET de productos abiertos al público ([AllowAnonymous]), pero exigir que para POST, PUT y DELETE el usuario esté autenticado con un token válido probado desde Swagger.